

Gestión de *workflows*

Exportación e importación de *workflows*

Exportar *workflows* Individuales desde la interfaz web

1. Abre el *workflow* en n8n.
2. Clic en menú (...) → "Download".
3. Se descarga el *workflow* como un archivo JSON.

Este archivo contiene toda la estructura del flujo: nodos, conexiones, configuraciones y parámetros. No incluye credenciales (usuarios y contraseñas de servicios externos), lo cual es correcto desde el punto de vista de la seguridad.



Exportación masiva desde la línea de comandos

Para exportar todos los *workflows* de una instancia (por ejemplo, para hacer un respaldo completo antes de una actualización) se puede usar la CLI de n8n dentro del contenedor Docker:

```
bash
docker exec -it nombre_del_contenedor n8n
export:workflow --all
--output=/home/node/.n8n/workflows-backup.json
```

Para exportar credenciales (con precaución, ya que contienen datos sensibles):

```
bash
docker exec -it nombre_del_contenedor n8n
export:credentials --all
--output=/home/node/.n8n/credentials-backup.json
```

Para copiar esos archivos desde el contenedor a la máquina local:

```
bash
docker cp
nombre_del_contenedor:/home/node/.n8n/workflows-backup.json ./
```

Importar *workflows*

Desde la interfaz web:

1. Se accede a n8n.
2. Clic en el menú de tres puntos (...).
3. *Import from file*, y se selecciona el archivo.

El *workflow* se importa con todos sus nodos y configuraciones, pero las credenciales deben reconectarse manualmente, ya que son específicas de cada instancia.

Desde línea de comandos:

```
bash
docker exec -i nombre_del_contenedor n8n
import:workflow
--input=/home/node/.n8n/workflows-backup.json
```

Estrategia de respaldos

Se recomienda **una rutina de respaldo semanal como mínimo para proyectos de clientes**. Este proceso puede automatizarse con un *workflow* de n8n que exporte los archivos y los envíe a Google Drive o a un *bucket* de almacenamiento en la nube.

Es un ejemplo concreto del poder de la herramienta: una automatización que respalda a la propia plataforma de automatizaciones.

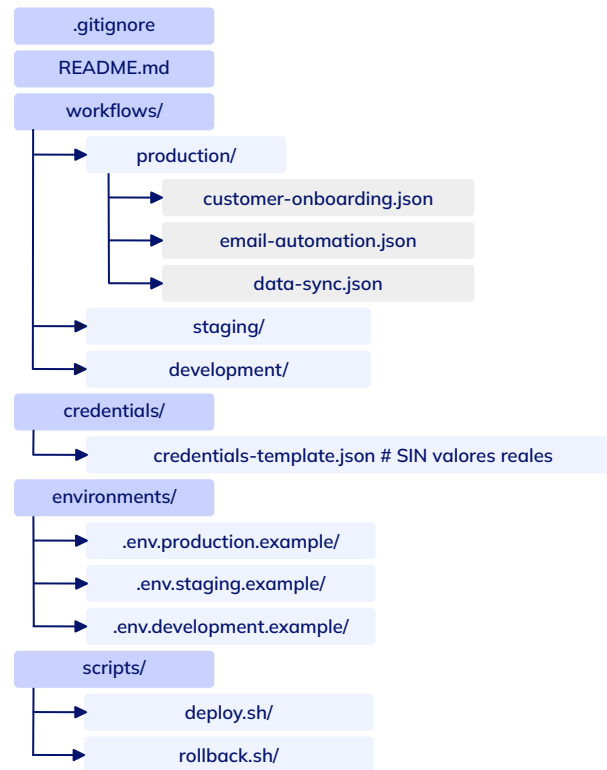
Versionado de *workflow* con Git

Cuando un *workflow* empieza a usarse en proyectos reales, es recomendable conservar distintas versiones del archivo JSON para saber qué cambió, cuándo cambió y poder volver a una versión anterior si algo deja de funcionar.

Git es una herramienta muy utilizada para registrar cambios en archivos. Los *workflows* exportados pueden organizarse en un repositorio, separados por cliente, entorno o función.

Estructura recomendada para repositorio:

n8n-workflows/



Buenas prácticas en producción

Usar PostgreSQL en lugar de SQLite

Por defecto, n8n almacena sus datos en SQLite, una base de datos de archivo que es excelente para desarrollo. Para producción con múltiples ejecuciones concurrentes o alta carga, PostgreSQL es significativamente más robusto, eficiente y confiable. El proceso de migración está documentado por n8n y es directo.

Para agregar PostgreSQL al stack de Docker Compose, se incluye el siguiente servicio en el `docker-compose.yml`:

```
yml
services:
  postgres:
    image: postgres:15
```


...

```
restart: always
environment:
  - POSTGRES_USER=${DB_POSTGRESDB_USER}
  - POSTGRES_PASSWORD=${DB_POSTGRESDB_PASSWORD}
  - POSTGRES_DB=${DB_POSTGRESDB_DATABASE}
volumes:
  - postgres_data:/var/lib/postgresql/data
```

```
n8n:
  # ... configuración de N8N ...
  depends_on:
    - postgres
```

```
volumes:
  postgres_data:
  n8n_data:
```

Nomenclatura de *workflows* y credenciales

Nombrar *workflows* con claridad

En producción puede haber decenas de *workflows* activos. Un nombre como "*Workflow 1*" o "*Test*" resulta inútil cuando algo falla. Conviene adoptar una convención de nombres descriptiva.

Por ejemplo:

[CLIENTE] Acción - Frecuencia, como [Acme]
Sincronizar pedidos Shopify → Sheets - Diario.

Usar credenciales con nombres descriptivos

n8n permite asignar nombres a las credenciales almacenadas. En lugar de "API Key", es preferible usar "Shopify - Cuenta producción Acme" o "Gmail - notificaciones@empresa.com".

Esto evita confusiones cuando se administran múltiples clientes en la misma instancia.

Separar entornos de desarrollo y producción

Una práctica fundamental es mantener dos instancias de n8n: **una para desarrollo y pruebas, y otra para producción.**

En desarrollo se puede experimentar sin riesgo de romper nada. En producción solo suben cambios ya probados. Esto es especialmente importante cuando los *workflows* de un cliente están siendo usados activamente.

Actualizaciones y reinicio

Actualizaciones controladas

Antes de actualizar n8n, siempre se realiza un respaldo. Las actualizaciones pueden introducir cambios de comportamiento que afecten *workflows* existentes. Después de actualizar, conviene revisar los *logs* y verificar que los *workflows* críticos sigan funcionando correctamente. **Para actualizar:**

```
bash
docker compose pull
docker compose up -d
```

Reinicio automático

La línea `restart: always` en el `docker-compose.yml` asegura que si n8n se detiene por cualquier motivo (fallo del servidor, reinicio del sistema), **vuelva a levantarse automáticamente sin intervención manual.**

Recursos adicionales

- [Documentación oficial de n8n](#)
- [n8n Community Forum](#)
- [n8n GitHub Repository](#)
- [Docker Documentation](#)
- [Prometheus Documentation](#)
- [Grafana Tutorials](#)
- [Let's Encrypt Documentation](#)

